



Computer Programming

CSC1261

Dr. Mukasine Angelique



Content

- **Chapter 1 Introduction**
- **Chapter 2 Data Types**
- **Chapter 3 Expressions and Operators**
- **Chapter 4 Decision Control Statement**
- **Chapter 5 Looping**
- **Chapter 6 Arrays**
- **Chapter 7 Functions**
- **Chapter 8 Structures**
- **Chapter 9 Pointers**



Chapter 1 Introduction

- 1.1 General Programming Concepts
- 1.2 Stages in Program Development
- 1.3 Debugging the Computer Program
- 1.4 C Fundamentals



1.1 GENERAL PROGRAMMING CONCEPTS



a. Programming Languages Description

- A **computer program** can be defined as a set of step-by-step computer instructions that are designed to tell a computer what and how to accomplish a task.
- **Computer programming** is the process of **designing, writing, testing, debugging**, and **maintaining** the source code of computer programs.
- A computer program's **source code** is the collection of files needed to **convert** from human-readable form to some kind of computer-executable form.
- The **source code** may be converted into an executable **file (object code)** by a compiler.



Programming Languages

- A **programming language** is an artificial language designed to express computations that can be performed by a machine, particularly a computer.
- Programming languages can be used to create programs that control the behavior of a machine, to express algorithms precisely, or as a mode of human communication.



Programming Languages categories

1. **Low level language**: is a programming language that is machine dependent, it runs on only on particular type of computer.
 - There are 2 types:
 - a. **Machine language**
 - Known as the first generation of programming languages, is the only language the computer recognizes.
 - Machine language is a collection of very detailed instructions that are meant to control the internal circuitry (hardware) of a particular computer.
 - Machine language instructions uses a series of binary (0's and 1's)



Programming Languages categories

b. Assembly language: the second generation of programming languages, a programmer writes instructions using symbolic codes, these are meaningful abbreviations and codes.

– e.g:

- ✓ A- for Addition
- ✓ C- for Compare
- ✓ L- for Load
- ✓ M- for Multiply



Programming language categories

2. High-level language

- A programming language such as C, FORTRAN, or Pascal that enables a programmer to write programs that are more or less independent of a particular type of computer.
- Such languages are considered high-level because they **are closer to human languages** and further from machine languages.
- In contrast, assembly languages are considered low-level because they are very close to machine languages.



Advantages of high level programming

1. The main advantage of high-level languages over low-level languages is that they are easier to read, write, and maintain.
2. Programs written in a high-level language must be translated into machine language by a compiler or interpreter.



Compilation Terminologies

- A program that is written in high-level language, must be translated into machine language before it can be executed.
- **Compilers**
 - The **compiler** translates the entire program (written in high-level language) into machine language before executing any set of the instructions.
 - A compiler reads the entire program and converts it to the **object code**. It provides errors not only for one line but for the entire program. Only errors free programs are executed.
 - It consumes little time for converting a **source program** to an **object program**. When the program length for any application is large, compilers are preferred.



Compilation Terminology

- Interpreter

- An interpreter reads only **one line of a source code** at a time and converts it to object codes.
- In case of any errors, the same will be indicated instantly.
- The program written with an interpreter can easily be read and understood by the other users as well.





Comparison between Interpreter and Compiler

Interpreter	Compiler
• It reads only one line of a source program at a time and converts it to object code.	• It reads the entire program and converts it to the object code.
• It is easier.	• It is not easier.
• It consumes more time for converting a source program to an object program.	• It consumes little time for converting a source program to an object program.



1.2 STAGES IN PROGRAM DEVELOPMENT

1. **Define or understand the problem:** First of all, you must understand a problem, if you don't know the problem you can't find a solution as desired output.
2. **Analyze the problem:** Once you know what the problem is, you can analyze it and make a plan to resolve it by thinking about the input data.
3. **Develop an algorithm and flowchart:** This is a process whereby a set of instructions are used to produce a solution to a given problem.
4. **Writing the code:** This is the next step where you write the codes for the program to make it work.
5. **Compiling and debugging the code:** Once the program coding is completed, you compile your program means you translate the source code to object code and if there are errors, you debug them. **Debugging** means to correct or remove errors (bugs) if there are in a program.
6. **Run the program:** To run an application means to execute it and check, if using some data, it is working with the correctness of the program



1.3 Debugging a Computer Program

- **Errors** are mistakes which we the programmers make.
- Programming errors are called **bugs**.
- The process of tracking them down and correcting them if they are in a computer program is therefore, called **Debugging**.
- **Programming errors come in 3 categories:**
 - ✓ Syntax error (compile error).
 - ✓ Run-time error.
 - ✓ Logic errors.



CONT'D...

*a. **Compile (syntax) error:** you get it when you've broken the rules of computer programming language.*

E.g: Spelling printf as prinntf or printif

- You also receive a compiler error, if you accidentally use the wrong punctuation or place a punctuation in the wrong place.

*b. **Run-time error:** it can be caused by attempting to do impossible arithmetic operations, such as calculating non-numeric data, dividing a number by zero, or find the square root of a negative number.*

*c. **Logic error:** with this, your application runs but produces incorrect result.*

- Perhaps the results of calculation are incorrect or the wrong text appears, or the text is ok but appears in the wrong location.



FLOWCHART

- A **flowchart**: A flow chart is a graphical or symbolic representation of a process.
- Each step in the process is represented by a different symbol and contains a short description of the process step.
- The flow chart symbols are linked together with arrows showing the process flow direction.
- And is also a graphical representation of an algorithm.

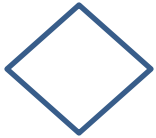


Different flow chart symbols

-  **Start/Terminator**: An oval flow chart shape indicating the start or end of the process.
-  **Data**: A parallelogram that indicates data input or output (I/O) for a process.
-  **Process**: A rectangular flow chart shape indicating a normal process flow step.



Different flow chart symbols



- **Decision**: A diamond flow chart shape indicating a branch in the process flow (Conditioning).

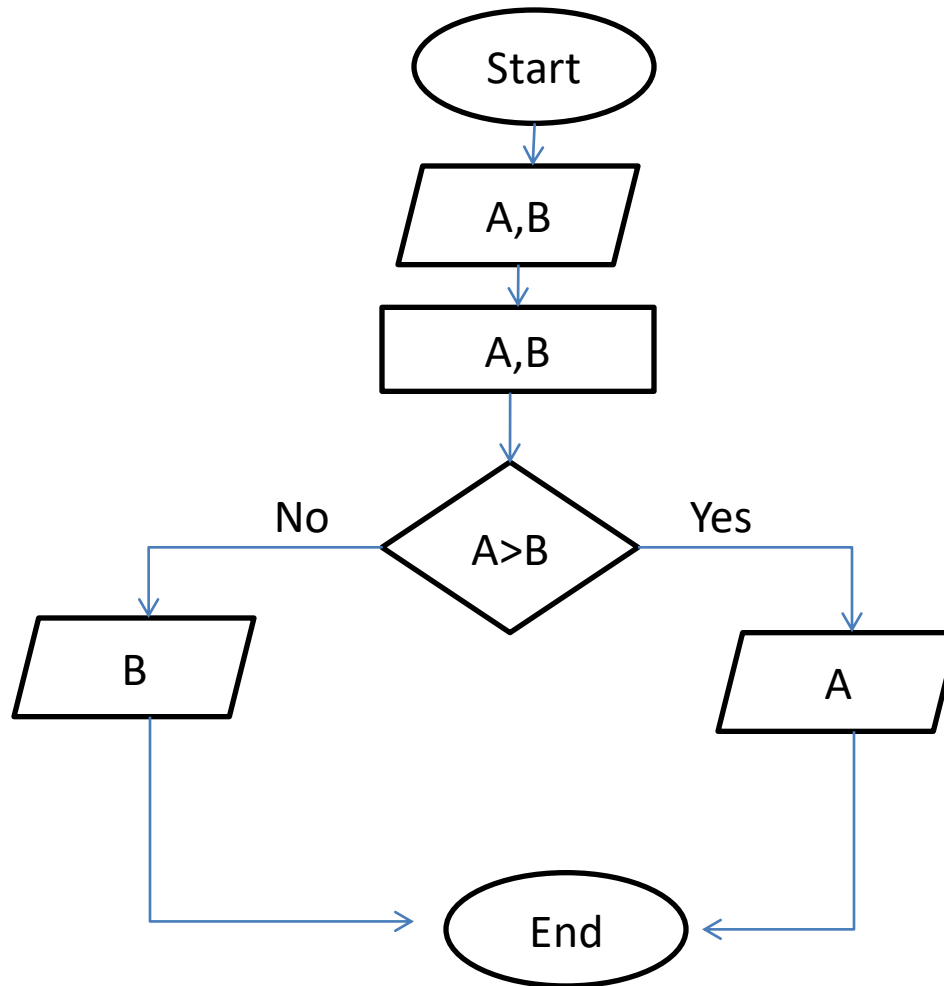


- **Connector**: A small, labeled, circular flow chart shape used to indicate a jump in the process flow



Example

- A flowchart to compare two numbers and display the greatest





1.4 C FUNDAMENTALS

- **C** is a computer programming language.
- **C** is the most popular programming languages developed in 1972 by Dennis Ritchie.
- **C** is one of thousands of programming languages currently in use.
- To write and run a C program, you must have access to a **C compiler**.



How C is applied in our daily basis

1. It is used in making computer games.
2. Used for developing operating systems.
3. Used in commercial company for robot control mechanism using micro controller system.
4. Used to control traffic light system. Using loop structures.
5. Used in motor system control for increase the speed and velocity of a motor.



Advantages of C

- C is portable
- C is Powerful and Flexible
- C has few keywords
- Wide acceptability (Popularity)



C is portable

- C program written for one computer system (an IBM PC, for example) can be compiled and run on another system with a little or no modification.
- Machine language or assembly language varies from computer to computer and hence, program written in these languages are not portable.



C is Powerful and Flexible

- What you can accomplish is limited only by your imagination. The language itself places no constraints on you.
- C is used for project as diverse as operating system, word processors, spreadsheet and even compiler of other language.
- C combines the convenience and portable nature of a high-level language with the flexibility of low-level language.



C has few keywords

- The following names are reserved by the C language.
- Their meaning is already defined, and they cannot be re-defined to mean anything else.

auto	break	case	char	continue	default
do	double	else	enum	extern	float
for	goto	if	int	long	register
return	short	sizeof	static	struct	switch
typedef	union	unsigned	void	volatile	while



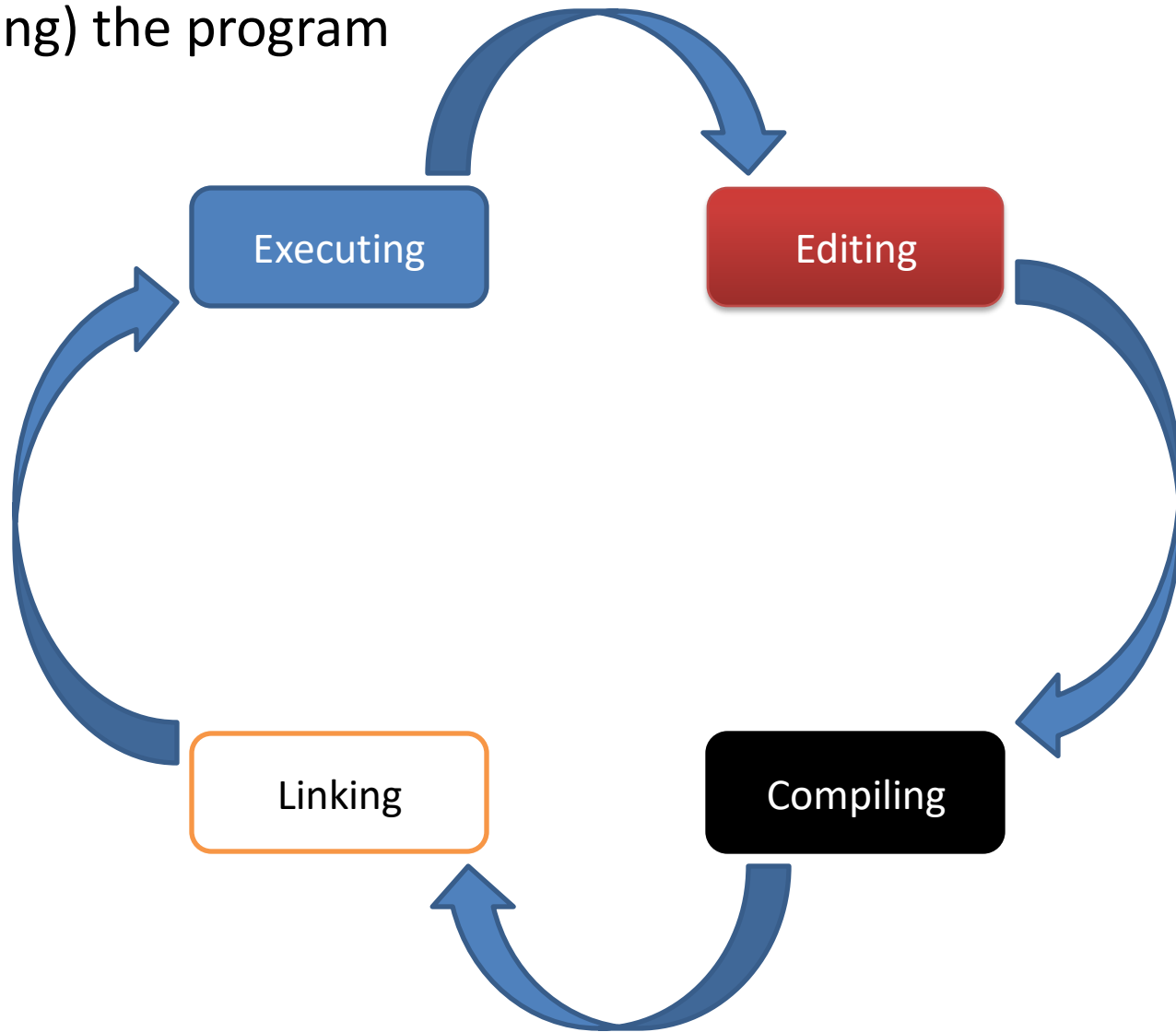
Wide acceptability (Popularity)

- C is a language known by the majority of programmers around the world.
- C is a popular language preferred by professional programmers, it has a powerful compiler.



Program Development Cycle

1. Editing(or writing) the program
2. Compiling it
3. Linking it
4. Executing it





Step 1

- Use an **editor to write** your source code(the program).
- By tradition C source code files have the extension .c
- That means that you have to save the source code with .c extension. ex: Hello.c



Step 2

- **Compile** the program using a compiler.
- If the compiler doesn't find any errors in the program, it produces an object file.
- The compiler produces object files with an .obj extension and the same name as the source code file (for example Hello.c to Hello.obj).

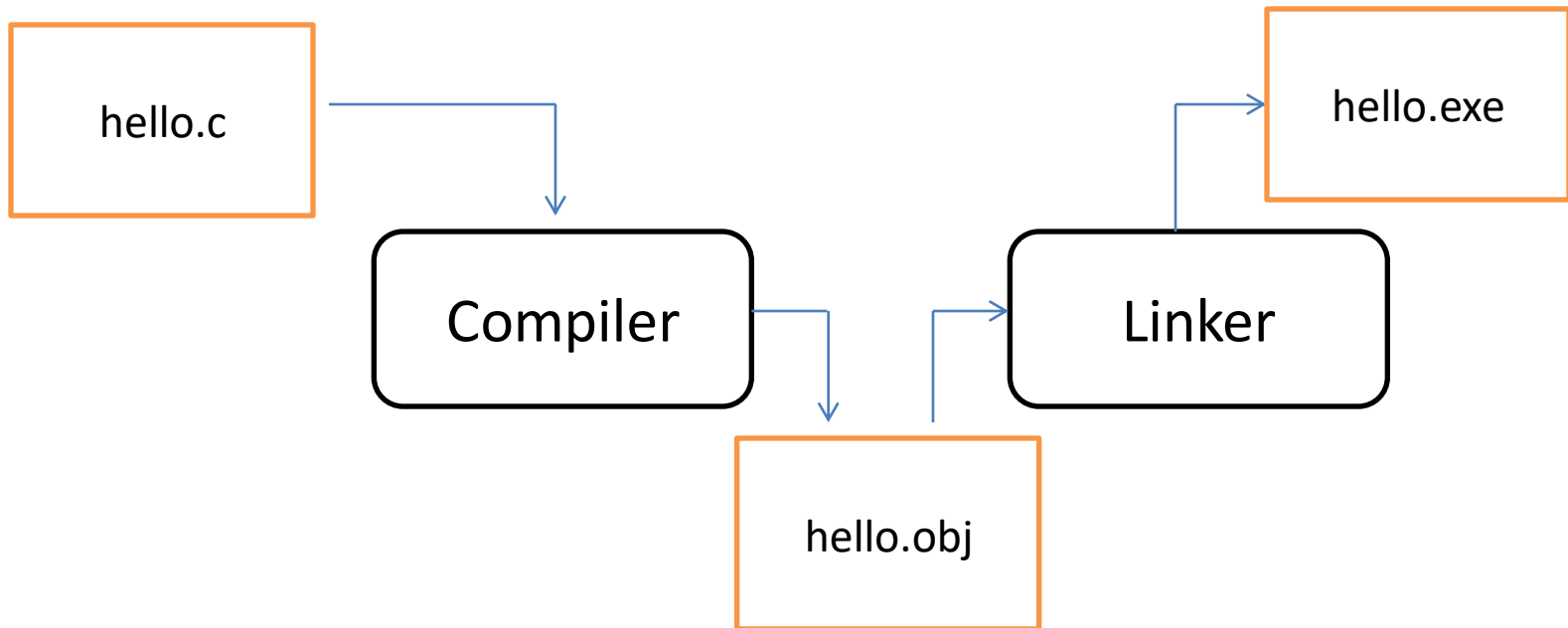


- If the compiler finds errors, it responds (reports) them, you must return to step 1 to make the corrections in your source code



Step 3

- **Link** the program with a Linker.
- If no errors occur, the Linker produces an executable program with an .exe extension and the same name as the object file.





Steps 4

- **Execute** the program to see the results.
- You should test to determine whether it functions properly, if no start again with step 1 and make modification and addition to your source code.



STRUCTURE OF A C PROGRAM

- Include header file section
- Global Declaration Section
- ***/* comments */***

main() Function name

{

/* comments*/

Declaration part

Executable part

}

User-defined functions{

}



STRUCTURE OF A C PROGRAM

- Consist of the following key parts:
 1. Include header file section.
 2. Global declaration.
 3. Function main.
 4. Declaration part.
 5. Executable part.
 6. User- defined function.
 7. Comments.



STRUCTURE OF A C PROGRAM

1. Include header file section: (Link Section)

- C program depends upon some **header files** for function definition that are used in program. Each **header file** by default is extended with **.h**. The file should be included using **# include directive** as given below.
- For example if you want to use some mathematical function then you have to define link for math.h file in link section.
- For Example
 - # include<stdio.h>**
 - # include<math.h>**



STRUCTURE OF A C PROGRAM

2. **Global declaration:** This section declares some variables that are used in more than one function. These variables are known as global variables. This section must be declared outside of all the functions.
3. **Function main:** Every program written in C language must contain main () function. Empty parentheses after main are necessary. **The function main () is a starting point of every 'C' program.** The execution of a program always begins with the function **main ()**. (A function is a self-contained block of statements that perform coherent task).
4. **The program execution starts from the opening brace ({) and ends with the closing brace (}).** Between these two braces the program should declare the declaration and the executable part.



5. Declaration part: The declaration part declares the entire variables that are used in executable part.

- The initialization of variables are also done in this section.
- The initialization means providing initial values to the variables.

6. Executable part: This part contains the statements following the declaration of the variables.

- It contains a set of statements or a single statement. These statements are enclosed between the braces. Eg. `printf(".....")`, `scanf("....")`



7. User-defined function: The functions defined by the user are called **user-defined functions**. These functions are generally defined after the **main ()** function. They can also be defined before **main ()** function.

8. Comments: Comments are not necessary in the program. However, to understand the flow of programs the programmer can include comments in the program.

Comments are used for documentation.

- Comments are nothing but some kind of statements which are placed between delimiters `/*` and `*/`.
- The compiler does not execute comments.



PROGRAMMING RULES

- A programmer while writing a program should follow the following rules:
 1. All statements should be written in **lower case letters**. Upper case letters are only used for **symbolic constants**.
 2. **Blank spaces** may be inserted between words to improve the readability of statements.
 3. It is not necessary to fix the position of **statement** in the program
 - I.e. the programmer can write the statement anywhere between the **two braces** following the declaration part. The user can also write one or more statements in one line separating them with a semicolon (;).

The following statements are valid.

- $a=b+c;$
- $d=b*c;$
- **Or** $a=b+c; d=b*c;$



4. The opening and closing braces should be balanced i.e. for example, if opening braces are **four**; then closing braces should also be **four**.
5. Every **C** statement should be ended with **Semi-colon (;)**



The Simplest C Program

```
#include <stdio.h>
main()
{
    printf("Hello!\n");
}
```

Program Output:

Hello!



Second Program

```
#include <stdio.h>  
main()  
{  
    printf("Hello, software\n");  
}
```

- Program Output:
Hello, software



Third program

```
#include <stdio.h>
```

```
main()
```

```
{
```

```
    printf("Hello,\n software\n development\n");
```

```
}
```

- Output

Hello,

software

development



Other special notation

- `\a` “bell” – i.e a beep
- `\b` Backspace
- `\f` Form Feed (new page)
- `\n` New Line (line feed)
- `\t` Tab
- `\v` Vertical Tab
- `\\` Backslash
- `\'` Single Quote
- `\"` Double Quote
- `\?` Question Mark



Comment example

```
#include<stdio.h>
main()
/* main function heading */
{
    printf("\n Hello, World! \n");
/* Display message on */
/* the screen */
}
```



Exercises

- Write a c program which will print the following out put :

- **Question 1:**

- Define the following word :
"programming"

- **Question 2:**

- $a=b+c$ a b c d
 - $b=a/5$



- **Question 3:**

- Write a C program to display your personal identification such as **Names, Registration number, Year of study, Department, Email, District of college location and telephone number.**



End
